

Navigation Centrale

Documentation du Projet

- [Accueil](#) | [Documentation Centralisée](#)

Démarrage Rapide

- [Guide Installation](#) | [Guide Développement](#)

API & Intégration

- [Documentation API](#) | [API Technique](#)

Déploiement

- [Guide Déploiement](#) | [Déploiement Technique](#)

Outils Développement

- [Scripts et Makefiles](#) | [Documentation Technique](#)

Documentation Technique

- [Architecture](#) | [Base de Données](#) | [Sécurité](#) | [Performance](#)
-

Déploiement JobbingTrack

Guide complet de déploiement de JobbingTrack en environnement de développement et production.

Vue d'Ensemble

JobbingTrack supporte plusieurs environnements de déploiement avec des configurations adaptées à chaque cas d'usage.

Environnements Supportés:

- | | |
|--|---------------------------|
| └─  Développement Local | (Docker Compose + Make) |
| └─  Production | (Docker Swarm/Kubernetes) |

└─ Cloud	(AWS/GCP/Azure)
└─ Enterprise	(On-premise avec orchestration)

Environnement de Développement

Prérequis

Matériel Requis

- **CPU** : 2+ cœurs (recommandé 4+)
- **RAM** : 4GB minimum (8GB recommandé)
- **Stockage** : 10GB disponible
- **Réseau** : Connexion internet stable

Logiciels Requis

```
# Docker et Docker Compose
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Node.js 20+ (pour développement frontend)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Git (pour le contrôle de version)
sudo apt-get install git

# Make (pour les commandes de build)
sudo apt-get install build-essential
```

Installation Rapide

```
# 1. Cloner le repository
git clone https://github.com/PavelDelhomme/JobbingTrack.git
cd JobbingTrack

# 2. Installation automatique
make install

# 3. Démarrage complet
make up
```

```
# 4. Vérification
make health
```

Structure de Développement

```

📁 JobbingTrack/
├── 📄 Makefile                # Commandes unifiées
├── 📁 backend/               # Microservices
├── 📁 frontend/              # Interface Next.js
├── 📁 scripts/                # Scripts d'automatisation
├── 📁 docs/                  # Documentation
└── 📁 tests/                  # Tests automatisés

```

Docker et Orchestration

Docker Compose (Développement)

Services Backend

```

version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: jobbingtrack
      POSTGRES_USER: jobbingtrack
      POSTGRES_PASSWORD: jobbingtrack123
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U jobbingtrack"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    command: redis-server --appendonly yes
    volumes:
      - redis_data:/data

  api-gateway:
    build: ./api-gateway

```

```

ports: ["3000:3000"]
depends_on:
  - postgres
  - redis
environment:
  - NODE_ENV=development

# Autres services...

```

Services Frontend

```

version: '3.8'
services:
  frontend:
    build: .
    ports: ["8080:3000"]
    environment:
      - NEXT_PUBLIC_API_URL=http://localhost:3000
      - NODE_ENV=development
    volumes:
      - ./app
      - /app/node_modules
      - /app/.next

```

Variables d'Environnement

Backend (.env)

```

# Base de données
DATABASE_URL=postgresql://jobbingtrack:jobbingtrack123@localhost:5432/jobbingtrack

# JWT
JWT_SECRET=your-super-secret-jwt-key-change-in-production
JWT_REFRESH_SECRET=your-refresh-secret-key

# Services
AUTH_SERVICE_URL=http://auth-service:3001
APPLICATION_SERVICE_URL=http://application-service:3002
COMPANY_SERVICE_URL=http://company-service:3003
CONTACT_SERVICE_URL=http://contact-service:3004
INTERVIEW_SERVICE_URL=http://interview-service:3005
NOTIFICATION_SERVICE_URL=http://notification-service:3006
DASHBOARD_SERVICE_URL=http://dashboard-service:3007

# Email (optionnel pour dev)

```

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-app-password
```

Frontend (.env.local)

```
NEXT_PUBLIC_API_URL=http://localhost:3000
NEXT_PUBLIC_DEBUG_AUTH=true
```

Déploiement en Production

Docker Swarm

Configuration Swarm

```
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    deploy:
      replicas: 1
      resources:
        limits:
          memory: 1G
        reservations:
          memory: 512M
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    deploy:
      replicas: 1
      resources:
        limits:
          memory: 512M

  api-gateway:
    image: jobbingtrack-api-gateway
    deploy:
      replicas: 2
      resources:
        limits:
```

```
memory: 256M
ports:
  - "3000:3000"
```

Déploiement

```
# Initialiser Swarm
docker swarm init

# Déployer la stack
docker stack deploy -c docker-compose.prod.yml jobbingtrack

# Surveiller
docker stack ps jobbingtrack
docker stack services jobbingtrack
```

Kubernetes (Alternative)

Manifests K8s

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-gateway
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api-gateway
  template:
    metadata:
      labels:
        app: api-gateway
    spec:
      containers:
        - name: api-gateway
          image: jobbingtrack-api-gateway:latest
          ports:
            - containerPort: 3000
          env:
            - name: NODE_ENV
              value: "production"
```

Déploiement Cloud

AWS (ECS + RDS)

Infrastructure as Code

```
# CloudFormation ou CDK
Resources:
  JobbingTrackCluster:
    Type: AWS::ECS::Cluster
    Properties:
      ClusterName: jobbingtrack-cluster

  Database:
    Type: AWS::RDS::DBInstance
    Properties:
      DBInstanceClass: db.t3.micro
      Engine: postgres
      EngineVersion: "15"
      DBName: jobbingtrack
      MasterUsername: jobbingtrack
      MasterUserPassword: "${DatabasePassword}"
```

Déploiement

```
# Construire les images
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin us-east-1.dkr.ecr.us-east-1.amazonaws.com

# Pousser les images
docker tag jobbingtrack-api-gateway:latest 123456789012.dkr.ecr.us-east-1.amazonaws.com/jobbingtrack-api-gateway:latest
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/jobbingtrack-api-gateway:latest

# Déployer
aws ecs update-service --cluster jobbingtrack-cluster --service api-gateway --force-new-deployment
```

Google Cloud Platform

Cloud Run + Cloud SQL

```
# Déployer sur Cloud Run
gcloud run deploy api-gateway \
  --image gcr.io/project-id/api-gateway \
```

```
--platform managed \
--region us-central1 \
--allow-unauthenticated

# Configurer Cloud SQL
gcloud sql instances create jobbingtrack-db \
--database-version=POSTGRES_15 \
--tier=db-f1-micro \
--region=us-central1
```



Déploiement Entreprise

Configuration On-Premise

Prérequis Infrastructure

- **Serveurs** : 3+ serveurs (app, db, cache)
- **Load Balancer** : Nginx ou HAProxy
- **Stockage** : NFS ou Ceph pour les volumes
- **Monitoring** : Prometheus + Grafana

Installation

```
# 1. Préparer les serveurs
ansible-playbook -i inventory.ini setup-servers.yml

# 2. Déployer l'application
ansible-playbook -i inventory.ini deploy-app.yml

# 3. Configurer le monitoring
ansible-playbook -i inventory.ini setup-monitoring.yml
```

Configuration Load Balancer

Nginx Configuration

```
upstream api_gateway {
    server api-1:3000;
    server api-2:3000;
    server api-3:3000;
}

server {
```



```
listen 80;
server_name api.jobbingtrack.com;

location / {
    proxy_pass http://api_gateway;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_cache_bypass $http_upgrade;
}
}
```

Configuration Avancée

SSL/TLS

Certificats Let's Encrypt

```
# Installation certbot
sudo apt-get install certbot

# Obtenir le certificat
sudo certbot certonly --webroot -w /var/www/html -d api.jobbingtrack.com

# Renouvellement automatique
sudo crontab -e
# Ajouter: 0 3 * * * certbot renew --quiet
```

Base de Données en Production

Configuration PostgreSQL

```
# postgresql.conf
max_connections = 200
shared_buffers = 256MB
effective_cache_size = 1GB
maintenance_work_mem = 64MB
checkpoint_completion_target = 0.7
wal_buffers = 16MB
```

```
default_statistics_target = 100
random_page_cost = 1.1
effective_io_concurrency = 200
work_mem = 1310kB
```

Optimisations Redis

```
# redis.conf
save 900 1
save 300 10
save 60 10000
appendonly yes
appendfsync everysec
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb
```



Monitoring et Observabilité

Prometheus Configuration

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'api-gateway'
    static_configs:
      - targets: ['api-gateway:3000']
    metrics_path: '/metrics'

  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres:9187']
```

Grafana Dashboards

Métriques Clés

- **Temps de réponse** moyen par endpoint
- **Taux d'erreur** par service
- **Utilisation CPU/Mémoire** par container
- **Nombre de connexions** actives à la base de données

- **Cache hit ratio** Redis

Sécurité en Production

Configuration SSL

```
server {  
    listen 443 ssl http2;  
    server_name api.jobbingtrack.com;  
  
    ssl_certificate /etc/letsencrypt/live/api.jobbingtrack.com/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/api.jobbingtrack.com/privkey.pem;  
  
    # Sécurité renforcée  
    ssl_protocols TLSv1.2 TLSv1.3;  
    ssl_ciphers ECDHE-RSA-AES128-GCM-SHA256:ECDHE-RSA-AES256-GCM-SHA384;  
    ssl_prefer_server_ciphers off;  
}
```

Secrets Management

Utilisation de Docker Secrets

```
# Créer les secrets  
echo "super-secret-jwt-key" | docker secret create jwt_secret -  
echo "database-password" | docker secret create db_password -  
  
# Utiliser dans docker-compose  
services:  
  api-gateway:  
    secrets:  
      - jwt_secret  
      - db_password
```

Maintenance et Mises à Jour

Stratégie de Déploiement

Blue-Green Deployment

```
# 1. Déployer la nouvelle version (green)
docker-compose -f docker-compose.blue.yml up -d

# 2. Tester la nouvelle version
curl http://localhost:3001/health

# 3. Basculer le trafic (nginx upstream)
# Modifier nginx.conf pour pointer vers les nouveaux containers

# 4. Arrêter l'ancienne version (blue)
docker-compose -f docker-compose.green.yml down
```

Backup et Restauration

Sauvegarde Automatique

```
# Script de sauvegarde quotidien
#!/bin/bash
DATE=$(date +%Y%m%d)
docker exec jobbingtrack_postgres pg_dump -U jobbingtrack jobbingtrack > backup_$(date +%Y%m%d).sql
aws s3 cp backup_$(date +%Y%m%d).sql s3://jobbingtrack-backups/
```

Restauration

```
# Restaurer depuis une sauvegarde
aws s3 cp s3://jobbingtrack-backups/backup_20250101.sql .
docker exec -i jobbingtrack_postgres psql -U jobbingtrack jobbingtrack < backup_20250101.sql
```



Scalabilité

Auto-scaling

Configuration Kubernetes HPA

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-gateway-hpa
```

```
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-gateway
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Load Balancing

Nginx Load Balancer

```
upstream api_backend {
    least_conn;
    server api-1:3000 weight=3;
    server api-2:3000 weight=2;
    server api-3:3000 weight=1;
}

server {
    listen 80;
    location / {
        proxy_pass http://api_backend;
        proxy_next_upstream error timeout invalid_header http_500 http_502 http_503;
    }
}
```



Résolution de Problèmes

Problèmes Courants

Service ne Démarre Pas

```
# Vérifier les logs
make logs-<service>
```

```
# Redémarrer le service
make restart-<service>

# Vérifier les ressources
docker stats
```

Base de Données Inaccessible

```
# Vérifier PostgreSQL
docker-compose exec postgres pg_isready

# Vérifier les connexions
docker-compose exec postgres psql -U jobbingtrack -c "SELECT 1"
```

Mémoire Insuffisante

```
# Vérifier l'utilisation mémoire
docker stats

# Redimensionner les containers
docker update --memory 512m <container>
```



Checklist Déploiement

Pré-déploiement

- ☐ Tests unitaires et d'intégration passent
- ☐ Tests E2E passent
- ☐ Code review effectué
- ☐ Documentation mise à jour
- ☐ Variables d'environnement configurées
- ☐ Backup de la base de données effectué

Déploiement

- ☐ Images Docker construites
- ☐ Services déployés
- ☐ Health checks passent
- ☐ Monitoring configuré
- ☐ Logs centralisés
- ☐ SSL configuré

Post-déploiement

- ☐ Tests de fumée effectués
- ☐ Métriques de performance vérifiées
- ☐ Backup automatique configuré
- ☐ Plan de rollback prêt

Support et Maintenance

Monitoring 24/7

- **Alertes** : Slack, email, SMS
- **Dashboards** : Grafana avec métriques temps réel
- **Logs** : Centralisation ELK Stack

Support Technique

- **Documentation** : Guides complets disponibles
- **Issues** : Suivi via GitHub Issues
- **Contact** : Support technique dédié

 **Déploiement JobbingTrack** - De la configuration locale à la production d'entreprise.

Navigation

-  [Index](#)
-  [Accueil](#)